Session 27 — Study Guide

What we built, and what's next

What we built during LIVE:

1. **A third real resource, the identical real pattern.** Comments reuse the exact same Server-Action-form + fetch-and-render shape issues and sprints already used — no new mechanism, only a new real endpoint.
2. **A cursor-aware `@mention` autocomplete.** Detecting an in-progress `@query` ending at the cursor (not anywhere in the text), searching the already-taught `GET /users/search` endpoint, and splicing a real `@[Name](id)` token in at the exact cursor position on selection.
3. **Rendering a stored mention token as a styled pill**, instead of raw markup — and why the token stores both the id (the backend's stable reference) and the display name (the frontend's rendering convenience).

What you will complete next:

- **TEAMWORK — a real, render-only per-project Activity feed.** The workspace-wide backend feed, filtered client-side to this project's own issues, the identical pattern Backlog and `SortableIssueList` already used.
- **HOMEWORK — a real Notifications center and comment deletion** (CORE), both mechanical transfer of `DeleteIssueButton`'s own already-taught shape; **the real cross-workspace Dashboard** (STRETCH), assembling data from two real, independent backend queries; plus the shared error boundary (STRETCH) and the `@mention` token design question (CHALLENGE).

The rest of this guide explains the complete Session 27 architecture — LIVE, TEAMWORK, and HOMEWORK together — so read it alongside the TEAMWORK and HOMEWORK tasks, not as a record of everything already built.

Why Session 27 needed it

Session 26 gave the app real Issue editing, a persisted board, and Sprints — but issues still had no way to discuss them, no record of who did what, and no way for a signed-in user to see their own work across every workspace they belong to. Session 27 closes that gap: a third real resource (comments), a real way to reference a teammate inside one, a real activity trail, a real place to see your own notifications, and a real personal dashboard. After completing LIVE + TEAMWORK + HOMEWORK, the required frontend curriculum is complete, ready for the Hackathon.

The problem, before the solution

Before today, an issue could be edited and moved across a board, but nobody could leave a note on it, reference a teammate directly, or see what had actually happened to it over time. There was also no single real place a signed-in user could check everything assigned to them, or everything they'd recently done, across every workspace at once.

The solution, step by step

Comment create/list, the same real shape as before. WHAT IS IT: `createCommentAction` + `CommentForm` (write) and a real fetch + `CommentList` (read) — the identical real Server-Action-form pattern `createIssueAction` / `CreateIssueForm` already established. WHY: a new resource earns no new pattern just because its own data is new; recognizing the shape is the actual skill, not re-deriving it from scratch. WHAT BREAKS WITHOUT IT: reinventing form-handling and fetch-and-render for every new resource, instead of recognizing an already-solved shape. HOW: `const action = createCommentAction.bind(null, issueId, basePath); useActionState(action, initialState)`. WHAT TO LOOK AT: `src/app/.../issues/[issueId]/CommentForm.tsx`, `CommentList.tsx`. WHAT TO EXPECT: posting a comment shows it in the list immediately, oldest first.

A cursor-aware `@mention` autocomplete. WHAT IS IT: a plain, uncontrolled-feeling `<textarea>` that detects an in-progress `@query` ending exactly at the cursor, searches real workspace members, and splices a token in on selection. WHY: a mention picker that searches the WHOLE text, or inserts at the wrong position, breaks the moment a comment has more than one `@` in it or the user isn't typing at the very end. WHAT BREAKS WITHOUT IT: picking a user could insert their token in the wrong place, or the dropdown could open for the wrong, already-finished mention. HOW: `TRAILING_MENTION_RE.exec(text.slice(0, cursor))` finds the in-progress query; `value.slice(0, mentionStart) + token + value.slice(cursor)` splices the real token back in. WHAT TO LOOK AT: `src/components/ui/MentionTextarea.tsx`. WHAT TO EXPECT: typing `@` then a few letters opens a real dropdown of matching workspace members; picking one inserts `@[Name](id)` exactly where you were typing.

Rendering a stored mention token. WHAT IS IT: a render-only component that finds every `@[Name](id)` token in a comment's own text and replaces it with a styled, non-interactive pill. WHY: the raw stored text should never reach the screen unprocessed — a reader shouldn't have to parse `@[Name](id)` syntax themselves. WHAT BREAKS WITHOUT IT: every comment would show its mentions as raw, unstyled markup instead of a real pill. HOW: a regex `exec` loop collects the text BEFORE each match, then the styled pill, repeating until no more matches, then appends whatever text is left after the last match. WHAT TO LOOK AT: `src/components/ui/MentionText.tsx`. WHAT TO EXPECT: a comment containing `@[Omar Rahal](2)` renders a real blue "@Omar Rahal" pill, never the raw text.

A real, per-project Activity feed. WHAT IS IT: fetch the workspace's own real, append-only activity log once, then keep only the entries whose issue belongs to this project — checked against a second, always-unfiltered fetch of every issue in this project, never against whatever the Issues list is currently filtered to. WHY: the real backend has no per-project activity route at all — it is intentionally workspace-wide — so a project-scoped VIEW of it is always a client-side filter. That filter must be based on ALL of this project's issues, because the visible, filterable Issues list and Activity membership are two separate things: filtering Issues down to "Open only" must never make another issue's Activity entries disappear. WHAT BREAKS WITHOUT IT: Project Detail would show every workspace's own activity, including entries about other projects entirely — or, if the filtered Issues list is used by mistake, Activity would incorrectly disappear whenever someone filters the Issues list. HOW: `const allIssuesRes = await apiFetch('/issues?projectId=' + project.id); const allIssues: Issue[] = await allIssuesRes.json(); const issueIds = new Set(allIssues.map((i) => i.id)); const activity = allActivity.filter((e) => e.issueId !== null && issueIds.has(e.issueId));`. WHAT TO LOOK AT:

`src/components/activity/ActivityFeed.tsx`, Project Detail's own `page.tsx`. WHAT TO EXPECT: the Activity section lists only this project's own real recent entries, and still shows entries for every issue in the project even when the Issues list itself is filtered.

Glossary (required)

- **Cursor-aware autocomplete:** a text-input helper that reads the caret's own position, not just the field's overall value, to decide what the user is "currently" typing.
- **Mention token:** the real stored shape `@[Display Name](integerUserId)` — carries both a stable reference (the id) and a display convenience (the name) in one piece of text.
- **Client-side filter over a broader fetch:** fetching more data than one view needs, then narrowing it down in the component itself, because the backend offers no narrower route.
- **Fire-and-forget Server Action:** a Server Action called from `onClick` without being `await`ed by the caller — its own `revalidatePath` call is what actually refreshes the page.

Common mistakes

- Computing the in-progress `@query` from the WHOLE textarea value instead of the text before the cursor — matches a stray `@` anywhere, not the one being typed right now.
- Forgetting to move the cursor past a newly inserted mention token.
- Re-resolving a stored mention token back to a live user object on the frontend before rendering it, instead of trusting `MentionText` to render the stored name directly.
- Comparing `entry.issueId` against a project's own id directly in the Activity filter — `issueId` identifies an issue, never a project.
- Building the Activity filter's issue-id set from the filtered `issues` list instead of a second, always-unfiltered fetch of this project's issues — then Activity entries for a real issue disappear the moment someone filters the Issues list to hide it.
- Giving `DeleteCommentButton` a client-side permission check before rendering, instead of letting it render unconditionally and the backend enforce the real rule.

Common errors and how to debug them

- **The mention dropdown never opens:** confirm the `@` was typed after a word boundary or at the very start of the text — `TRAILING_MENTION_RE` deliberately requires that, so a real email-like `foo@bar` never triggers it.
- **A posted mention renders as raw text, not a pill:** confirm the comment's own stored `body` actually contains the real token shape `@[Name](id)` (check the real `POST` request's payload in devtools), before assuming `MentionText`'s own regex is at fault.
- **The Activity section on Project Detail is empty:** confirm `GET /workspaces/:workspaceId/activity` itself actually returns rows for `curl`, before assuming the client-side project filter removed everything.
- **A real `403` on comment delete:** confirm the signed-in user is actually this comment's own author or a workspace admin/owner — `requireCanMutate` rejected it correctly.

Never "fix" any of these by adding a client-side mention-validation shortcut, or by removing the backend's own author-or-admin check on comment mutation.

Review questions (required)

1. Why does `MentionTextarea` compute its in-progress query from `value.slice(0, cursor)`, instead of the full `value`?
2. Why does a real mention token store both the display name and the id?
3. Why does `ActivityFeed` filter entries client-side instead of asking the backend for a per-project route?
4. Why does `DeleteCommentButton` render for every comment, with no client-side permission check?

Answers

1. Scoping to the text before the cursor finds the `@` the user is typing next to right now, not an unrelated, already-finished mention earlier in the same comment.
2. The id is the backend's own stable reference (names can collide or change); the name lets the frontend render instantly from the comment's own stored text, with no extra network call.
3. The real backend's own activity route is intentionally workspace-wide — there is no narrower route to ask for, so the narrowing happens in the component itself, using a second, always-unfiltered fetch of this project's issues (never the filtered Issues list) to decide which activity entries belong to this project.
4. It mirrors `DeleteIssueButton`'s own established convention: the backend enforces authorization independently and rejects an unauthorized attempt with a real `403`.

What you should be able to explain now (required)

- Why recognizing an already-taught pattern (the Server-Action-form shape) matters more than memorizing a specific resource's own syntax.
- Why a cursor-aware text helper needs the caret's own position, not just the field's value.
- Why a mention token's own stored shape serves two different real jobs at once.
- Why a client-side filter is sometimes the correct choice, not a workaround, when the backend genuinely offers no narrower route.

Not yet (required)

- Epics, sub-task hierarchies, burndown/velocity charts, custom workflows (explicitly excluded from this program's scope). Deeper real-time features (e.g. live-updating notifications without a reload) remain out of scope for the required curriculum.